

MATERI PEMBELAJARAN

Tipe Data Bentukan

(Derived & User-Defined Data Types) dalam C++

Pemrograman C++ | Tingkat Menengah

Tipe data bentukan (*derived & user-defined types*) adalah tipe data yang dibentuk dari tipe data dasar maupun dari tipe bentukan lain. C++ menyediakan berbagai mekanisme untuk membangun tipe data yang lebih kompleks dan bermakna sesuai kebutuhan program.

Peta Tipe Data Bentukan C++:

Kategori	Tipe Data	Keterangan Singkat
Derived Types	Array	Kumpulan elemen bertipe sama
Derived Types	Pointer	Menyimpan alamat memori
Derived Types	Reference	Alias/nama lain sebuah variabel
Derived Types	Function	Tipe fungsi (tipe kembalian & parameter)
User-Defined	struct	Kumpulan data berbeda tipe
User-Defined	union	Berbagi satu lokasi memori
User-Defined	enum	Konstanta bernama bertipe integer
User-Defined	class	Struct + method + akses kontrol (OOP)

Kategori	Tipe Data	Keterangan Singkat
Standard Library	string	Rangkaian karakter (std::string)

1. Array

Array adalah kumpulan elemen dengan **tipe data yang sama** yang disimpan secara berurutan di memori. Setiap elemen diakses menggunakan indeks yang dimulai dari **0**.

Deklarasi Array:

```
tipe_data nama_array[ukuran];
tipe_data nama_array[ukuran] = {nilai1, nilai2, ...};
```

Array 1 Dimensi:

```
#include <iostream>
using namespace std;

int main() {
    int nilai[5] = {85, 90, 78, 92, 88};

    // Akses elemen
    cout << "Elemen ke-0 : " << nilai[0] << endl; // 85
    cout << "Elemen ke-4 : " << nilai[4] << endl; // 88

    // Iterasi dengan for loop
    int total = 0;
    for (int i = 0; i < 5; i++) {
        total += nilai[i];
    }
    cout << "Rata-rata : " << (double)total / 5 << endl;
    return 0;
}
```

Array 2 Dimensi (Matriks):

```

int matriks[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// Akses: matriks[baris][kolom]
cout << matriks[1][2]; // output: 6 (baris 1, kolom 2)

// Cetak matriks
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        cout << matriks[i][j] << "t";
    }
    cout << endl;
}

```

■ Indeks array dimulai dari 0, bukan 1. Mengakses indeks di luar batas (out-of-bounds) adalah Undefined Behavior yang berbahaya — C++ tidak memberikan error otomatis!

2. Pointer

Pointer adalah variabel khusus yang menyimpan **alamat memori** dari variabel lain. Pointer adalah fitur paling kuat sekaligus paling berisiko di C++ jika tidak digunakan dengan benar.

Operator	Nama	Fungsi
&	Address-of	Mengambil alamat memori sebuah variabel
*	Dereference	Mengakses nilai di alamat yang ditunjuk pointer
->	Arrow	Akses member melalui pointer ke struct/class

Contoh Dasar Pointer:

```

#include <iostream>
using namespace std;

int main() {
    int angka = 42;
    int* ptr = &angka; // ptr menyimpan alamat 'angka'

    cout << "Nilai angka : " << angka << endl; // 42
    cout << "Alamat angka : " << &angka << endl; // misal: 0x61ff08
    cout << "Nilai ptr : " << ptr << endl; // sama dengan &angka
    cout << "Nilai *ptr : " << *ptr << endl; // 42 (dereference)

    // Mengubah nilai melalui pointer
    *ptr = 100;
    cout << "angka sekarang : " << angka << endl; // 100
    return 0;
}

```

Pointer dan Array:

```

int arr[4] = {10, 20, 30, 40};
int* p = arr; // nama array = alamat elemen pertama

cout << *p; // 10 (elemen ke-0)
cout << *(p+1); // 20 (elemen ke-1, pointer arithmetic)
cout << *(p+3); // 40 (elemen ke-3)

// Iterasi dengan pointer
for (int i = 0; i < 4; i++) {
    cout << *(p + i) << " ";
}

```

Dynamic Memory Allocation:

```
#include <iostream>
using namespace std;

int main() {
    // Alokasi memori di heap
    int* p = new int(99); // alokasi 1 integer
    int* arr = new int[5]; // alokasi array 5 integer

    *p = 77;
    for (int i = 0; i < 5; i++) arr[i] = i * 10;

    cout << "*p = " << *p << endl;

    // WAJIB: bebaskan memori setelah selesai
    delete p;
    delete[] arr;
    p = nullptr; // good practice: set ke nullptr setelah delete
    arr = nullptr;
    return 0;
}
```

■ Setiap new harus diimbangi dengan delete. Lupa delete menyebabkan memory leak. Dalam C++ modern, lebih disarankan menggunakan smart pointer (`unique_ptr`, `shared_ptr`) dari header `<memory>`.

3. Reference (Referensi)

Reference adalah alias (nama lain) dari sebuah variabel yang sudah ada. Berbeda dengan pointer, reference **tidak bisa diubah** untuk merujuk ke variabel lain setelah diinisialisasi, dan **tidak bisa bernilai null**.

Aspek	Pointer	Reference
Sintaks	<code>int* p = &x;</code>	<code>int& r = x;</code>
Bisa null?	Ya (<code>nullptr</code>)	Tidak
Bisa diubah arah?	Ya	Tidak
Perlu dereference?	Ya (<code>*p</code>)	Tidak (langsung <code>r</code>)
Penggunaan utama	Memori dinamis, array	Parameter fungsi, alias

```
#include <iostream>
using namespace std;

void tambahSatu(int& n) { // parameter by reference
    n++; // mengubah variabel asli
}

int main() {
    int x = 10;
    int& r = x; // r adalah alias dari x

    r = 50;
    cout << "x = " << x << endl; // 50 (x ikut berubah!)

    tambahSatu(x);
    cout << "x = " << x << endl; // 51
    return 0;
}
```

■ Reference paling sering digunakan sebagai parameter fungsi untuk menghindari penyalinan data besar (pass by reference) dan memungkinkan fungsi mengubah nilai asli.

4. Struct (Struktur)

Struct memungkinkan pengelompokan beberapa variabel dengan **tipe data berbeda** menjadi satu unit logis. Struct sangat berguna untuk merepresentasikan entitas dunia nyata.

Sintaks Dasar:

```
struct NamaStruct {
    tipe1 anggota1;
    tipe2 anggota2;
    // ...
}; // <-- jangan lupa titik koma!
```

Contoh Lengkap:

```

#include <iostream>
#include <string>
using namespace std;

struct Mahasiswa {
    string nama;
    int nim;
    float ipk;
    char jeniskelamin;
};

void cetakMahasiswa(const Mahasiswa& mhs) {
    cout << "Nama : " << mhs.nama << endl;
    cout << "NIM : " << mhs.nim << endl;
    cout << "IPK : " << mhs.ipk << endl;
}

int main() {
    Mahasiswa mhs1 = {"Budi Santoso", 12345678, 3.75f, 'L'};
    Mahasiswa mhs2;
    mhs2.nama = "Siti Rahayu";
    mhs2.nim = 87654321;
    mhs2.ipk = 3.90f;

    cetakMahasiswa(mhs1);
    cetakMahasiswa(mhs2);

    // Array of struct
    Mahasiswa kelas[3] = {
        {"Andi", 111, 3.5f, 'L'},
        {"Bety", 222, 3.8f, 'P'},
        {"Ciko", 333, 3.2f, 'L'},
    };
    cout << "IPK Bety: " << kelas[1].ipk << endl;
    return 0;
}

```

Struct dengan Pointer (Linked List Sederhana):

```

struct Node {
int data;
Node* next; // pointer ke node berikutnya
};

Node* buatNode(int nilai) {
Node* baru = new Node;
baru->data = nilai;
baru->next = nullptr;
return baru;
}

```

■ Akses member struct menggunakan titik (.) untuk objek langsung, dan tanda panah (->) jika melalui pointer ke struct.

5. Union

Union mirip struct, tetapi semua anggota **berbagi lokasi memori yang sama**. Hanya satu anggota yang dapat menyimpan nilai valid pada satu waktu. Ukuran union ditentukan oleh anggota terbesar.

```

#include <iostream>
using namespace std;

union Data {
int i;
float f;
char c;
}; // ukuran = max(sizeof(int), sizeof(float), sizeof(char)) = 4 byte

int main() {
Data d;

d.i = 42;
cout << "int : " << d.i << endl; // 42 (valid)

d.f = 3.14f;
cout << "float : " << d.f << endl; // 3.14 (valid)
cout << "int : " << d.i << endl; // TIDAK VALID (d.i ditimpa d.f)

cout << "Ukuran union: " << sizeof(Data) << " byte" << endl; // 4
return 0;
}

```

Perbedaan Struct vs Union:

Aspek	Struct	Union
Memori	Setiap anggota memiliki memori sendiri	Semua anggota berbagi memori

Aspek	Struct	Union
Ukuran	Jumlah ukuran semua anggota	Ukuran anggota terbesar
Data aktif	Semua anggota bisa aktif sekaligus	Hanya 1 anggota aktif sekaligus
Penggunaan	Data record/entitas	Hemat memori, variant data

■ Union sering digunakan dalam pemrograman sistem/embedded untuk menghemat memori atau untuk interpretasi ulang bit data (type punning).

6. Enum (Enumerasi)

Enum mendefinisikan sekumpulan **konstanta bernama** bertipe integer. Enum membuat kode lebih mudah dibaca dibandingkan menggunakan angka mentah (magic numbers).

Enum Klasik (C-style):

```
#include <iostream>
using namespace std;

enum HariKerja {
    SENIN = 1, // nilai dimulai dari 1
    SELASA, // otomatis 2
    RABU, // otomatis 3
    KAMIS, // otomatis 4
    JUMAT // otomatis 5
};

enum Warna { MERAH, HIJAU, BIRU }; // default mulai dari 0

int main() {
    HariKerja hari = RABU;
    cout << "Hari ke-" << hari << endl; // 3

    if (hari == RABU) cout << "Hari ini Rabu" << endl;

    Warna cat = HIJAU;
    cout << "Kode warna: " << cat << endl; // 1
    return 0;
}
```

Enum Class (C++11, lebih aman):

```
enum class Arah { UTARA, SELATAN, TIMUR, BARAT };
enum class Status { AKTIF, NONAKTIF, PENDING };

int main() {
    Arah tujuan = Arah::TIMUR; // wajib pakai scope ::
    Status akun = Status::AKTIF;

    if (tujuan == Arah::TIMUR) {
        cout << "Menuju Timur" << endl;
    }

    // Konversi ke int (perlu explicit cast)
    int kode = static_cast<int>(tujuan); // 2
    cout << "Kode arah: " << kode << endl;
    return 0;
}
```

■ *Gunakan enum class (C++11) daripada enum biasa. enum class mencegah konflik nama dan konversi implisit yang tidak disengaja ke integer.*

7. String (std::string)

std::string adalah tipe data dari Standard Library C++ untuk menyimpan rangkaian karakter (teks). Berbeda dengan C-string (array char), std::string jauh lebih aman dan mudah digunakan.

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    string nama = "Budi Santoso";
    string kota = "Makassar";
    string gabung = nama + " dari " + kota; // concatenation

    cout << gabung << endl;
    cout << "Panjang : " << nama.length() << endl; // 12
    cout << "Huruf ke-0: " << nama[0] << endl; // 'B'
    cout << "Uppercase? : " << (nama == "BUDI SANTOSO") << endl;

    // Substr
    string depan = nama.substr(0, 4); // "Budi"
    cout << "Substr : " << depan << endl;

    // Cari posisi
    int pos = nama.find("Santoso");
    cout << "'Santoso' di posisi: " << pos << endl; // 5

    // Replace
    nama.replace(5, 7, "Prasetyo");
    cout << "Setelah replace: " << nama << endl;
    return 0;
}

```

Method Penting std::string:

Method / Operator	Fungsi	Contoh
length() / size()	Panjang string	s.length()
+ atau +=	Penggabungan string	s1 + s2
[i]	Akses karakter ke-i	s[0]
substr(pos, len)	Ambil bagian string	s.substr(2, 4)
find(sub)	Cari posisi substring	s.find("abc")
replace(pos,n,str)	Ganti bagian string	s.replace(0,3,"XYZ")
empty()	Cek apakah kosong	s.empty()
erase(pos, n)	Hapus n karakter dari pos	s.erase(2, 3)
append(str)	Tambah di akhir	s.append(" world")
compare(str)	Bandingkan dua string	s.compare("abc")

8. Class (Pengantar OOP)

Class adalah perluasan dari struct yang mendukung **enkapsulasi**, **metode (fungsi anggota)**, dan **kontrol akses** (public, private, protected). Class adalah fondasi Pemrograman Berorientasi Objek (OOP) di C++.

Aspek	Struct	Class
Akses default	public	private
Method	Bisa (C++)	Bisa (utama)
Inheritance	Bisa	Bisa (umum digunakan)
Penggunaan	Data sederhana	OOP, abstraksi kompleks

Contoh Class:

```

#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private: // hanya bisa diakses dalam class
string pemilik;
double saldo;

public: // bisa diakses dari luar
// Constructor
BankAccount(string nama, double saldoAwal) {
pemilik = nama;
saldo = saldoAwal;
}

void setor(double jumlah) {
if (jumlah > 0) saldo += jumlah;
}

bool tarik(double jumlah) {
if (jumlah > saldo) { cout << "Saldo tidak cukup!" << endl; return false; }
saldo -= jumlah;
return true;
}

void info() const {
cout << "Pemilik: " << pemilik << ", Saldo: " << saldo << endl;
}
};

int main() {
BankAccount rek("Budi", 1000000.0);
rek.setor(500000);
rek.tarik(200000);
rek.info(); // Pemilik: Budi, Saldo: 1300000
return 0;
}

```

■ Materi class dan OOP (inheritance, polymorphism, encapsulation) akan dibahas lebih mendalam pada modul Pemrograman Berorientasi Objek (OOP) C++.

9. typedef dan using (Type Alias)

C++ menyediakan dua cara untuk membuat alias (nama alternatif) bagi tipe data yang sudah ada, sehingga kode lebih ringkas dan mudah dibaca.

```

#include <iostream>
#include <vector>
using namespace std;

// Cara lama: typedef
typedef unsigned long long ULL;
typedef int Matriks[4][4];

// Cara baru (C++11): using – lebih jelas dan fleksibel
using Bilangan = double;
using VectorInt = vector<int>;
using FungsiOp = int(*)(int, int); // alias untuk tipe fungsi

int tambah(int a, int b) { return a + b; }

int main() {
    ULL besar = 9000000000000000000ULL;
    Bilangan pi = 3.14159;
    VectorInt daftar = {1, 2, 3, 4, 5};
    FungsiOp op = tambah;

    cout << "ULL : " << besar << endl;
    cout << "pi : " << pi << endl;
    cout << "op : " << op(10, 20) << endl; // 30
    return 0;
}

```

■ Gunakan 'using' (C++11) daripada 'typedef' karena sintaksnya lebih intuitif dan mendukung template alias.

10. Ringkasan & Panduan Memilih

Kebutuhan	Gunakan
Kumpulan elemen bertipe sama	Array / std::vector
Alamat memori / memori dinamis	Pointer (atau smart pointer)
Alias variabel, parameter by-reference	Reference (&)
Kumpulan data berbeda tipe (entitas)	struct
Berbagi memori, hemat ruang	union
Konstanta bernama / pilihan terbatas	enum class
Data teks / string	std::string
Data + perilaku + enkapsulasi (OOP)	class
Nama alternatif tipe data	using / typedef

Hubungan antar tipe: struct dapat berisi array, pointer ke struct lain (linked list), bahkan class. Pointer dan array saling berkaitan erat di level memori. Memahami tipe data bentukan adalah kunci untuk merancang struktur data dan algoritma yang efisien.

Selamat belajar C++ lebih lanjut!

Modul berikutnya: Fungsi & Rekursi — Pemrograman C++